

Witness-Generated Regular Split Trees for ALCI RCC5

Option 2: A Self-Contained Decidability Proof

May 2026

Abstract

We give a self-contained decidability proof for concept satisfiability in ALCI.RCC5 over strong abstract complete atomic RCC5 frames. The proof uses a containment-oriented split-tree architecture (Option 2). From any satisfying abstract Kripke/RCC5 model we restrict to a witness-generated submodel and introduce selected witness-cover edges. The split presentation uses weak-equality splitting, typed congruence quotienting, transitive eventuality acceptance via parity, and support-closed sibling mosaics. We show: a concept is satisfiable iff it has a valid finite witness-generated regular split-tree certificate; these certificates are effectively enumerable and checkable.

Contents

1	The result	2
2	Abstract RCC5 semantics	2
3	Closure and types	2
4	Witness-generated submodels	3
5	Generated covers and containment orientation	3
6	Finite profiles and transitive eventualities	5
6.1	Summary update and eventuality monitor semantics	5
7	Parity automaton encoding for transitive eventualities	7
7.1	Alphabet and eventuality sets	7
7.2	Automaton definition	7
7.3	Acceptance semantics	8
8	Correctness of the automaton encoding	8
9	Lasso lemma and ultimately periodic trunks	9
10	Triple support lemma for mosaics	9
11	Typed congruence and quotient correctness	10
11.1	Computing typed congruence	10
12	Finite certificates	12

13 Soundness	12
14 Completeness	13
15 Decision procedure	13
16 Test formulas and worked example	14
16.1 Chain formula	14
17 Implementation note	15
18 Concluding remarks	15
A RCC5 composition table	15
B Worked certificate examples	16
B.1 Example 1: Upward chain (already sketched)	16
B.2 Example 2: Sibling triple requiring triple support	17

1 The result

Theorem 1.1 (Main theorem). *Concept satisfiability for $ALCI_RCC5$ over strong abstract $RCC5$ frames is decidable. More precisely, for every concept C_0 in negation normal form,*

$$C_0 \text{ is satisfiable} \iff C_0 \text{ has a valid finite witness-generated regular split-tree certificate.}$$

2 Abstract $RCC5$ semantics

Let $B = \{EQ, DR, PO, PP, PPI\}$ be the $RCC5$ base relations. The inverse operation inv is defined by $inv(EQ) = EQ$, $inv(DR) = DR$, $inv(PO) = PO$, $inv(PP) = PPI$, $inv(PPI) = PP$. Composition is given by the standard $RCC5$ composition table (use the usual table in the formal write-up).

Definition 2.1 (Strong abstract $RCC5$ frame). *A strong abstract $RCC5$ frame is a pair (D, p) with $D \neq \emptyset$ and $p : D \times D \rightarrow B$ satisfying:*

- (F1) $p(x, x) = EQ$ for all $x \in D$;
- (F2) $p(y, x) = inv(p(x, y))$ for all $x, y \in D$;
- (F3) $p(x, z) \in comp(p(x, y), p(y, z))$ for all $x, y, z \in D$.

An interpretation adds unary predicates for concept names and evaluates $ALCI$ constructors in the standard way, with role atoms interpreted by the $RCC5$ relation predicate $p(\cdot, \cdot) = R$.

3 Closure and types

Fix an input concept C_0 in negation normal form. Let $Cl(C_0)$ be the least finite set containing $C_0, \top, \perp$, closed under subconcepts, Boolean decomposition, and modal duals: if $\forall R.D \in Cl(C_0)$ then $\exists R.D \in Cl(C_0)$ and vice versa.

Definition 3.1 (Hintikka type). A type $T \subseteq \text{Cl}(C_0)$ is a set satisfying:

- $\top \in T, \perp \notin T$;
- for each $D \in \text{Cl}(C_0)$ exactly one of $D, \neg D$ is in T ;
- conjunctions/disjunctions are respected.

The finite set of all types is $\text{Type}(C_0)$.

For a type T and relation R define $\text{Need}_R(T) = \{D \in \text{Cl}(C_0) \mid \forall R.D \in T\}$ (the formulas that universals require of R -neighbours).

4 Witness-generated submodels

Definition 4.1 (Closure witness choice). Let $\mathcal{I} = (D, p, \cdot^{\mathcal{I}})$ be a strong *RCC5* interpretation. For each $a \in D$ and each existential closure formula $\exists R.D \in \text{Cl}(C_0)$ with $a \in (\exists R.D)^{\mathcal{I}}$ choose one witness $w(a, \exists R.D) \in D$ such that $p(a, w(a, \exists R.D)) = R$ and $w(a, \exists R.D) \in D$.

Definition 4.2 (Witness-generated subdomain). Given $a_0 \in D$, define $W_0 = \{a_0\}$ and $W_{n+1} = W_n \cup \{w(a, \exists R.D) \mid a \in W_n, \exists R.D \in \text{Cl}(C_0), a \in (\exists R.D)^{\mathcal{I}}\}$. Let $W = \bigcup_{n \geq 0} W_n$ and consider the restricted interpretation $\mathcal{I}|_W$.

Lemma 4.3 (Witness-generated submodel lemma). For every $a \in W$ and every $D \in \text{Cl}(C_0)$,

$$a \in D^{\mathcal{I}} \iff a \in D^{\mathcal{I}|_W}.$$

In particular, if $a_0 \models C_0$ in $\mathcal{I}$ then $a_0 \models C_0$ in $\mathcal{I}|_W$.

Proof. By induction on the structure of D . Atomic and Boolean cases are immediate. For $D = \exists R.E$, if $a \in (\exists R.E)^{\mathcal{I}}$ then the chosen witness $w(a, \exists R.E) \in W$ and by induction $w(a, \exists R.E) \in E^{\mathcal{I}|_W}$, so $a \in (\exists R.E)^{\mathcal{I}|_W}$. Conversely, any witness in the restriction is a witness in the original model. For $D = \forall R.E$, if $a \in (\forall R.E)^{\mathcal{I}}$ then every R -neighbour in W is also an R -neighbour in $\mathcal{I}$ and hence satisfies E ; conversely, if $a \notin (\forall R.E)^{\mathcal{I}}$ then there is a counter-witness b with $p(a, b) = R$ and $b \notin E^{\mathcal{I}}$; since $\exists R.\neg E \in \text{Cl}(C_0)$ the witness selection will include a counter-witness in W , so the universal is false in the restriction as well. $\square$

Remark 4.4. The generated witness edges are immediate in the generated presentation but are not claimed to be immediate semantic Hasse covers of the full *RCC5* proper-part order. This avoids density/Hasse issues.

5 Generated covers and containment orientation

We orient generated containment edges in the containment-oriented way: parent (superpart) above, child (proper part) below. A single semantic element may have several selected superpart witnesses; the split construction creates multiple occurrences (weak-EQ mates) for that element, each with a single parent in the occurrence tree.

Definition 5.1 (Selected containment demand). Given a witness-generated model $J = \mathcal{I}|_W$, a selected containment demand is a pair (y, x) with x PP y arising from a generated witness $y \xrightarrow{\text{gen}} x$ or $x \xrightarrow{\text{gen}} y$.

Definition 5.2 (Containment occurrence tree). *A containment occurrence tree is a countable set T with a partial parent map $\text{par} : T \rightarrow T$ such that the parent graph is acyclic on occurrences. If $\text{par}(t) = s$ then the vertical semantic label is PP from the child to the parent (or PPI conversely). Parent chains may be finite, one-sided infinite, or bi-infinite around the evaluation occurrence. No ambient top or bottom is assumed.*

Definition 5.3 (Weak split presentation). *A weak split presentation of J consists of:*

- *a containment occurrence tree T ,*
- *a projection $\pi : T \rightarrow W$,*
- *an equivalence relation $=$ on occurrences whose classes are contained in projection fibres (weak-EQ mates),*
- *a complete atomic labelling $\text{PT} : T \times T \rightarrow B$.*

It must satisfy:

- (S1) *if $\text{par}(t) = s$ then $\text{PT}(t, s) = \text{PP}$ and conversely;*
- (S2) *every selected containment demand (y, x) is realized by some occurrence t of x and parent occurrence s of y ; if another occurrence of x already exists the new one is marked as a weak-EQ mate;*
- (S3) *$\text{PT}(t, u) = p_J(\pi(t), \pi(u))$ for extracted presentations, or satisfies the same finite RCC5 constraints in abstract certificates.*

Lemma 5.4 (Split-tree cover lemma). *Every witness-generated model J has a weak containment split-tree presentation in which each occurrence has only finitely many generated witness obligations, bounded by a function of $|\text{Cl}(C_0)|$.*

Proof. Construct occurrences by fair expansion: start with an occurrence projecting to the evaluation element; when an occurrence t projecting to x has a selected PPI witness $x \xrightarrow{\text{gen}} z$ add a child occurrence u with $\pi(u) = z$; when t has a selected PP witness $x \xleftarrow{\text{gen}} y$ create a fresh occurrence t' with $\pi(t') = x$, mark $t' = t$, and place t' below an occurrence s with $\pi(s) = y$. Repeat for all generated witness requirements. Irreflexivity and transitivity of PP prevent parent cycles. Each occurrence has finitely many active witness demands because $\text{Cl}(C_0)$ is finite. $\square$

Definition 5.5 (Typed congruence). *A weak split presentation has typed congruence if $t = t'$ and $u = u'$ imply $\text{tp}(t) = \text{tp}(t')$ and $\text{PT}(t, u) = \text{PT}(t', u')$.*

Lemma 5.6 (Strong quotient). *A weak split presentation with typed congruence quotients to a strong RCC5 frame on the quotient domain with pair labelling induced by PT.*

Proof. Typed congruence makes PT independent of representatives. Weak-EQ occurs only inside $=$ -classes, so EQ becomes identity after quotienting. Converse and composition are inherited from the weak labelling. $\square$

6 Finite profiles and transitive eventualities

A finite certificate represents the split tree by finitely many profiles and one-hole contexts. Profiles record local types, summaries, eventuality monitors, equality colours, and sibling status.

Definition 6.1 (Profile). *A profile is a tuple*

$$p = (\text{tp}(p), \text{Le}(p), \text{Ly}(p), \text{M}(p), e(p), s(p)),$$

where $\text{tp}(p) \in \text{Type}(C_0)$, $\text{Le}, \text{Ly} \subseteq \text{Cl}(C_0)$ are lower/upper summaries, $\text{M}(p) \subseteq \mathcal{E}$ is the finite eventuality monitor set, $e(p)$ is an equality colour, and $s(p) \in \{\text{core}, \text{front}, \text{out}\}$ is sibling status.

Remark 6.2 (Inverse role obligations). *Profiles and summaries explicitly record obligations arising from inverse roles. Concretely, if a type contains a formula of the form $\exists R^-.D$ or $\forall R^-.D$ (where R^- denotes the inverse of R), then the profile's $\text{M}(p)$ and the lower/upper summaries include the corresponding requirements so that `update_summary` and local mosaics can check and propagate inverse obligations along the trunk and into side contexts. In practice the `newLowerC` and `newUpperSafeC` tables are computed using both forward and inverse obligations recorded in contexts, so inverse constraints are treated symmetrically in the mosaic closure rules.*

6.1 Summary update and eventuality monitor semantics

We formalize the finite functions used by the trunk automaton: the summary update function `update_summary` and the eventuality monitor update `update_monitor`. Both are finite table functions computable from the certificate alphabet.

Definition 6.3 (Lower and upper summaries). *A lower summary Le and an upper summary Ly are elements of $\mathcal{P}(\text{Cl}(C_0))$. We write $S_{\text{sum}} = \mathcal{P}(\text{Cl}(C_0)) \times \mathcal{P}(\text{Cl}(C_0))$ for the finite set of summary pairs (Le, Ly) .*

Definition 6.4 (Eventuality monitor). *Let $\mathcal{E}$ be the finite set of transitive eventualities appearing in $\text{Cl}(C_0)$. A monitor valuation is a function $m : \mathcal{E} \rightarrow \{0, 1, 2\}$ where*

- $m(\eta) = 0$ means “not pending”,
- $m(\eta) = 1$ means “pending and not yet seen in the current lap”,
- $m(\eta) = 2$ means “pending and seen at least once since last reset”.

The finite set of valuations is denoted $M_{\text{mon}} = \{0, 1, 2\}^{\mathcal{E}}$.

Definition 6.5 (Lower and upper update tables). *For each one-hole context $C \in \text{Ctx}_Q$ and for each pair of profiles $p, p' \in \Sigma$ that C may connect, the certificate records two finite sets $\text{newLower}_C(p, p') \subseteq \text{Cl}(C_0)$ and $\text{newUpperSafe}_C(p, p') \subseteq \text{Cl}(C_0)$. They satisfy:*

- $\text{newLower}_C(p, p')$ contains exactly those closure formulas that the context C guarantees to hold at some strict lower occurrence introduced when p' is plugged under p .
- $\text{newUpperSafe}_C(p, p')$ contains exactly those closure formulas that remain true everywhere in the extended lower region after the plug.

Define

$$\text{newLower}(p, p') = \bigcup_{C \text{ compatible}} \text{newLower}_C(p, p'), \quad \text{newUpperSafe}(p, p') = \bigcap_{C \text{ compatible}} \text{newUpperSafe}_C(p, p'),$$

where the unions/intersections range over the finite set of contexts in the certificate that may connect p and p' . These unions and intersections are finite because the certificate contains finitely many contexts.

Lemma 6.6 (Summary soundness). *Let a trunk be built using contexts compatible with the certificate and let summaries be updated by `update_summary` using the recorded tables. Then for every occurrence in the trunk:*

1. *every formula $D \in \text{Le}$ is witnessed in the strict lower region represented below that occurrence; and*
2. *every formula $D \in \text{Ly}$ holds everywhere in the represented lower region.*

Proof. By induction on the construction of the trunk. Each context C that plugs a profile p' under p contributes exactly the formulas in $\text{newLower}_C(p, p')$ as witnessed formulas in the newly represented strict lower region; hence the union $\text{newLower}(p, p')$ records all such witnesses. Similarly, $\text{newUpperSafe}_C(p, p')$ records formulas that no context introduces a counterexample for; the intersection therefore records formulas that remain true everywhere. The finite certificate tables guarantee the local witness and safety checks used in the induction step. $\square$

Lemma 6.7 (Universals enforced by summaries and local checks). *Let $\forall R.D \in \text{Cl}(C_0)$ occur in a profile p on a trunk whose transitions use the certificate tables. Suppose the trunk is built using contexts and mosaics from the certificate and summaries are updated by `update_summary`. Then:*

1. *If every R -neighbour shape reachable from p via the certificate's contexts and mosaics records D in its profile or in the appropriate summary, no transition rejects on this universal.*
2. *Conversely, if some finite prefix exhibits an R -neighbour that violates D , then the corresponding transition is rejecting.*

Proof. Contexts and mosaics in the certificate enumerate all local R -neighbour shapes that can appear adjacent to an occurrence with profile p . The update tables $\text{newLower}_C, \text{newUpperSafe}_C$ and the local mosaic checks are constructed precisely to (i) record witnesses for positive existentials and (ii) detect counterexamples to universals. Hence if every reachable shape records D in its profile or summary, the local checks never fire a rejecting transition. Conversely, any finite counterexample appears in some finite prefix; the local checks detect it and the transition rejects. $\square$

Update rules Define two total functions

$$\text{update_summary} : S_{\text{sum}} \times \Sigma \times \Sigma \rightarrow S_{\text{sum}}, \quad \text{update_monitor} : M_{\text{mon}} \times \Sigma \times \Sigma \rightarrow M_{\text{mon}}.$$

Intuitively, the triple (s, p, p') means: current summaries s , current profile p , next profile p' (the context between them is encoded in the certificate and determines the update table).

Concrete finite table definition.

1. `update_summary`((`Le`, `Ly`), p, p') is computed by:

- $\text{Le}' = \text{Le} \cup \text{newLower}(p, p')$, where $\text{newLower}(p, p') \subseteq \text{Cl}(C_0)$ is the finite set of closure formulas that the certificate records as realized in the strict lower region introduced by plugging p' under p .
- $\text{Ly}' = \text{Ly} \cap \text{newUpperSafe}(p, p')$, where $\text{newUpperSafe}(p, p')$ is the finite set of formulas that remain true everywhere in the extended lower region after the plug.

Both `newLower` and `newUpperSafe` are finite certificate tables computed from contexts and mosaics.

2. `update_monitor`(m, p, p') is computed by:

- For each $\eta \in \mathcal{E}$:
 - If $\eta \in \text{M}(p)$ then set $m'(\eta) := \max(m(\eta), 1)$.
 - If $p' \in F_\eta$ then set $m'(\eta) := 2$ (seen).
 - If a universal safety check at p requires immediate failure (certificate records a violation), then the transition is rejecting.
 - Otherwise, if $m(\eta) = 2$ and the certificate records an explicit reset point (for example, completion of a generator loop or an annotated reset in the context), set $m'(\eta) := 0$; otherwise keep $m'(\eta) := m(\eta)$.

Lemma 6.8. *The functions `update_summary` and `update_monitor` are computable by finite tables derived from the certificate. Moreover, along an upward trunk `Le` is monotone nondecreasing and `Ly` is monotone nonincreasing; symmetric statements hold for downward trunks.*

Proof. Both functions consult only finite certificate tables (contexts, mosaics, and profile components). The monotonicity follows from the definitions of `newLower` and `newUpperSafe`: lower summaries only accumulate witnessed closure formulas; upper summaries only lose formulas when a counterexample appears. $\square$

7 Parity automaton encoding for transitive eventualities

This section gives a formal deterministic parity automaton that recognizes exactly the legal infinite trunks required by a valid certificate.

7.1 Alphabet and eventuality sets

Fix the finite closure $\text{Cl}(C_0)$ and the finite profile alphabet Σ used in certificates. Each profile $p \in \Sigma$ is a tuple as above.

Let $\mathcal{E}$ be the finite set of transitive eventualities that appear in $\text{Cl}(C_0)$, i.e. formulas of the form $\exists^+ R.D$ (with $R \in \{\text{PP}, \text{PPI}\}$) and their duals. For each $\eta \in \mathcal{E}$ define the finite acceptance set

$$F_\eta = \{p \in \Sigma \mid D \in \text{tp}(p) \text{ or summaries guarantee } D\}.$$

7.2 Automaton definition

Define the deterministic parity automaton $\mathcal{P} = (Q, \Sigma, q_0, \delta, \pi)$.

States Let S_{sum} be the finite set of possible summary vectors and $M_{\text{mon}} = \{0, 1, 2\}^{\mathcal{E}}$ the monitor valuations. Set $Q = \Sigma \times S_{\text{sum}} \times M_{\text{mon}}$. The initial state $q_0 = (p_0, s_0, m_0)$ is determined by the certificate.

Transition function The transition function $\delta : Q \times \Sigma \rightarrow Q$ is deterministic and defined by finite table lookups. Given $(p, s, m) \in Q$ and next profile p' :

1. compute $s' = \text{update_summary}(s, p, p')$;
2. compute $m' = \text{update_monitor}(m, p, p')$; if a universal safety check fails the transition is rejecting;
3. set $\delta((p, s, m), p') = (p', s', m')$.

Parity priorities Partition $\mathcal{E}$ by syntactic nesting depth and translate the family $\{F_\eta\}$ into a parity condition by a standard lexicographic encoding. This yields a finite priority function $\pi : Q \rightarrow \{0, \dots, k\}$.

Lemma 7.1 (Parity correctness). *Let $\mathcal{P} = (Q, \Sigma, q_0, \delta, \pi)$ be the deterministic parity automaton constructed above with priority function π obtained by the lexicographic encoding of the family of Büchi sets $\{F_\eta\}_{\eta \in \mathcal{E}}$. An infinite run of $\mathcal{P}$ satisfies the parity acceptance condition iff for every eventuality $\eta \in \mathcal{E}$ that is pending infinitely often along the run, the set F_η is visited infinitely often.*

Proof. The construction of π groups eventualities by syntactic nesting depth and encodes the finite monitor vector into parity ranks in lexicographic order. Concretely, for each nesting level the monitor bits for eventualities at that level are encoded into a finite integer rank; the parity index is the concatenation (sum) of these ranks with offsets so that higher nesting levels dominate lower ones.

If the run visits some eventuality η 's pending state infinitely often but visits F_η only finitely often, then the monitor bit for η remains pending on the tail and the corresponding parity component attains an odd value infinitely often; hence the minimum priority seen infinitely often is odd and the parity condition fails.

Conversely, if every eventuality pending infinitely often is also satisfied (i.e., its F_η is visited infinitely often), then for each nesting level the encoded monitor vector stabilizes to values that ensure the minimum priority seen infinitely often is even. Therefore the parity condition holds.

This is the standard lexicographic Büchi→parity correctness argument: the parity indices are chosen so that the parity acceptance is equivalent to the conjunction of the Büchi conditions for all $\eta \in \mathcal{E}$. $\square$

7.3 Acceptance semantics

A run $q_0, q_1, \dots$ is accepting iff the minimum priority occurring infinitely often is even. This enforces that no transitive eventuality remains perpetually pending.

8 Correctness of the automaton encoding

Lemma 8.1 (Soundness). *If a trunk satisfies the certificate safety and eventuality conditions, then $\mathcal{P}$ has an accepting run on that trunk.*

Proof. Safety conditions are local checks; the transition function rejects only when a safety condition fails. Since the trunk satisfies safety, no rejecting transition occurs. For each transitive eventuality η required at some occurrence, the trunk contains a later occurrence in the required direction whose profile lies in F_η . The monitor marks η pending and clears it when F_η is seen. The parity priorities are chosen to be equivalent to the Büchi family, so the run satisfies parity acceptance. $\square$

Lemma 8.2 (Completeness). *If $\mathcal{P}$ accepts an infinite trunk, then the trunk satisfies the certificate safety and eventuality conditions.*

Proof. Acceptance implies no rejecting transition occurred, hence safety holds. The parity acceptance implies that every eventuality monitor bit that is set infinitely often is also cleared infinitely often, so every pending eventuality is discharged. Therefore the trunk meets the certificate eventuality requirements. $\square$

9 Lasso lemma and ultimately periodic trunks

Lemma 9.1 (Lasso lemma). *If there exists an infinite trunk accepted by $\mathcal{P}$, then there exists an ultimately periodic trunk accepted by $\mathcal{P}$.*

Proof. $\mathcal{P}$ is deterministic with finite state set Q . Consider an accepting run $q_0, q_1, \dots$. The run eventually enters a strongly connected component $C \subseteq Q$ and remains there. Because the run is accepting, the parity condition is satisfied on the tail inside C . Choose a cycle in C that visits all states required by the parity acceptance sets; such a cycle exists because C is strongly connected and intersects the acceptance sets as required. Let the prefix up to the first visit of C be the stem and the chosen cycle be the loop. Repeating the loop yields an ultimately periodic trunk whose run follows the same state cycle and therefore satisfies the parity condition. Safety and summary updates are preserved because transitions are deterministic table lookups. $\square$

10 Triple support lemma for mosaics

Definition 10.1 (Support-closed mosaic set). *Let $\mathcal{M}$ be a finite set of atomic mosaics (finite RCC5 networks over profile endpoints). We say $\mathcal{M}$ is support-closed if it satisfies the following closure conditions:*

- (SC1) **Submosaic closure:** *every submosaic (obtained by deleting endpoints and restricting labels consistently) of a mosaic in $\mathcal{M}$ is in $\mathcal{M}$.*
- (SC2) **Pairwise support closure:** *whenever $\mathcal{M}$ contains two mosaics that share an endpoint and their pairwise edge labels are compatible, then any two-endpoint mosaic induced on the shared endpoints is in $\mathcal{M}$.*
- (SC3) **Triple support closure:** *whenever $\mathcal{M}$ contains pairwise mosaics realizing relations R_{12}, R_{13} on endpoints 1–2 and 1–3, and $R_{23} \in \text{comp}(R_{12}, R_{13})$, then $\mathcal{M}$ contains a three-endpoint mosaic realizing R_{12}, R_{13}, R_{23} (when relation-safety and typed-EQ consistency permit).*

Lemma 10.2 (Triple support). *Let $\mathcal{M}$ be a support-closed mosaic set. Suppose profiles p_1, p_2, p_3 occur in the certificate alphabet and that pairwise relations R_{12}, R_{13} are permitted by $\mathcal{M}$. Then $\mathcal{M}$ contains a three-endpoint mosaic realizing simultaneously R_{12}, R_{13} and some $R_{23} \in \text{comp}(R_{12}, R_{13})$.*

Proof. By the triple support closure rule in the construction of $\mathcal{M}$ the closure algorithm explicitly adds such a three-endpoint mosaic whenever the pairwise mosaics exist and composition permits a compatible third relation. $\square$

Lemma 10.3 (Finite closure of mosaic set). *Let Σ be the finite profile alphabet determined by $\text{Cl}(C_0)$. The iterative closure procedure that adds submosaics, pairwise supports, and triple-support mosaics to an initial finite seed of mosaics terminates after finitely many steps and yields a finite support-closed mosaic set $\mathcal{M}$.*

Proof. All mosaics are finite RCC5 networks over the finite set of profile endpoints and labels drawn from a finite set. Each closure step only produces mosaics whose labels and endpoints are drawn from these finite sets. Hence the universe of possible mosaics is finite, so the closure process must stabilise after finitely many additions. $\square$

Case analysis for RCC5 composition To make the triple-support closure fully explicit we list the nontrivial composition cases used in the construction. Let $R_{12}, R_{13} \in B$ be the pairwise relations realized by two mosaics sharing endpoint 1. The possible $R_{23} \in \text{comp}(R_{12}, R_{13})$ are given by the standard RCC5 composition table. The certificate closure rule adds a three-endpoint mosaic for each R_{23} in the composition set.

For reference, the relevant composition outcomes (representative cases) are:

(R_{12}, R_{13})	$\text{comp}(R_{12}, R_{13})$
(PP, PP)	{PP, PO, DR}
(PP, PPI)	{PO, DR, EQ}
(PO, PO)	{PO, DR}
(DR, DR)	{DR}
(PP, PO)	{PO, DR}

(Full table is standard; the certificate only needs the finite subset that actually arises from the profile alphabet.)

Construction remark For each listed R_{23} the triple-support closure constructs a three-vertex mosaic by merging the two pairwise mosaics along the shared endpoint and filling the third edge with R_{23} . Relation-safety and typed-EQ consistency are checked locally; if a conflict arises the composition case is discarded. Because the universe of atomic mosaics is finite, this process is effective and exhaustive for the certificate alphabet.

11 Typed congruence and quotient correctness

11.1 Computing typed congruence

We give a finite algorithm that computes the equivalence relation $\sim$ used for quotienting and prove it is a typed congruence.

Algorithm Compute $\sim$ on the finite set of occurrences as the least equivalence relation satisfying closure rules. Start with base relation:

$$t \sim_0 u \iff e(t) = e(u) \text{ and } \text{tp}(t) = \text{tp}(u).$$

Iterate: if $t \sim_n t'$ and $u \sim_n u'$ and for every third occurrence v there exists v' with $v \sim_n v'$ and $PT(t, v) = PT(t', v')$, then add $t \sim_{n+1} t'$. Close under reflexivity, symmetry, transitivity. Terminate when stable; finiteness ensures termination.

Lemma 11.1 (Typed congruence algorithm correctness). *The relation $\sim$ computed above is an equivalence relation and satisfies: if $t \sim t'$ and $u \sim u'$ then $PT(t, u) = PT(t', u')$. Hence $\sim$ is a typed congruence suitable for quotienting.*

Proof. Termination and equivalence follow from finiteness and closure. For the congruence property, by construction for every third occurrence v there is a matching v' with $v \sim v'$ and identical recorded pair labels; typed-EQ synchronization and support-closure guarantee joint witnesses when needed. Thus pairwise labels are representative-independent. $\square$

Lemma 11.2 (Finite prefix realization). *Let T be the unfolding of a certificate and let S be any finite set of occurrences in T together with the pairwise recorded labels between them. Then there exists a finite connected prefix $P \subseteq T$ that contains S and all intermediate occurrences and contexts needed to witness the recorded pair labels among elements of S . Consequently, any finite set of occurrences and the contexts connecting them appear in some finite prefix of the unfolding.*

Proof. By construction the unfolding records pair labels only when they are witnessed by finite contexts/mosaics. For each recorded pair between occurrences in S collect the finite sets of occurrences and contexts used to record that pair; the union of these finite sets is a finite connected prefix P containing S and the required witnesses. $\square$

Lemma 11.3 (Global composition lifting). *Let $[t], [u], [v]$ be three quotient classes in the quotient constructed from a valid certificate. Then there exist representatives t', u', v' occurring in some finite prefix of the unfolding such that $PT(t', u')$, $PT(u', v')$, and $PT(t', v')$ are recorded and satisfy the RCC5 composition constraints. Consequently composition holds for the classes.*

Proof. Let $[t], [u], [v]$ be quotient classes and pick representatives $t_0 \in [t]$, $u_0 \in [u]$, $v_0 \in [v]$ from the unfolding. By construction of the unfolding and by support-closure of the mosaic set, there exist finite contexts and mosaics that connect these occurrences: specifically, there is a finite connected subgraph (a finite prefix) of the unfolding that contains t_0, u_0, v_0 and all intermediate occurrences and contexts used to record the pair labels $PT(t_0, u_0)$ and $PT(u_0, v_0)$. By Lemma 11.2 we can realize this finite subgraph using mosaics from the support-closed set $\mathcal{M}$.

Within this finite prefix the RCC5 composition constraint is enforced by the local mosaic checks: the mosaic (or combination of mosaics) that witnesses the two pairwise relations yields a recorded label for the third pair (t', v') consistent with $\text{comp}(PT(t', u'), PT(u', v'))$. Because the prefix is finite, the required three-endpoint mosaic (or its induced label) appears explicitly in the prefix or in the support closure of the mosaics used. Finally, typed congruence (Lemma 11.1) ensures that the recorded labels lift to the quotient classes $[t], [u], [v]$ and are independent of the chosen representatives. Hence composition holds for the classes. $\square$

Theorem 11.4 (Quotient correctness). *Let T be the unfolding of a valid certificate with typed congruence $\sim$. Quotienting occurrences by $\sim$ yields a strong RCC5 interpretation $\mathcal{I}$ in which every closure formula in $\text{Cl}(C_0)$ has the same truth value at the image of the initial occurrence as in the unfolding.*

Proof. Define domain $D = \{[t]_\sim : t \in T\}$. Define $p([t]_\sim, [u]_\sim) = PT(t, u)$ (well-defined by Lemma 11.1). Contexts and mosaics ensure local composition and converse; Lemma 11.2 lifts local checks to arbitrary finite prefixes. Existential witnesses are provided by contexts/mosaics,

lower/upper summaries, or parity-accepted trunk positions; universals are preserved because all R -neighbours of a representative are represented by occurrences whose images are the R -neighbours of the quotient element. Induction on closure formulas yields the preservation of truth. $\square$

12 Finite certificates

Definition 12.1 (Regular split-tree certificate). *A certificate for C_0 is a finite tuple*

$$Q = (\text{Prof}_Q, \text{Ctx}_Q, \text{Mos}_Q, G_Q, p_*, \Pi_Q)$$

where Prof_Q is a finite profile set, Ctx_Q a finite set of one-hole containment contexts, Mos_Q a finite support-closed mosaic set, G_Q a finite generator graph, p_* an initial profile with $C_0 \in \text{tp}(p_*)$, and Π_Q a parity condition for transitive eventualities. Cycles in G_Q are blocking cycles that unfold to fresh occurrences.

A certificate is *valid* when the following finite checks pass:

- all types are Hintikka types;
- all contexts and mosaics are finite RCC5 networks satisfying composition and converse;
- relation-safety holds on all edges;
- lower/upper summaries are coherent with contexts;
- PP/PPI universals propagate as required;
- PP/PPI existentials satisfy parity acceptance via Π_Q ;
- DR/PO existentials have explicit witnesses in contexts/mosaics;
- equality colours define a typed congruence and mosaics are support-closed;
- the finite generator has an accepting unfolding from p_* .

13 Soundness

Theorem 13.1 (Soundness). *Every valid finite witness-generated regular split-tree certificate unfolds to a strong RCC5 model satisfying C_0 .*

Proof. Unfold the finite generator, creating fresh occurrences on every traversal of a blocking cycle. By Lemma 11.2 and support-closure the unfolding has a complete weak RCC5 labelling satisfying composition, relation-safety, and typed congruence. Quotient weak equality using Lemma 5.6; the result is a strong RCC5 interpretation. Induction on closure formulas (as in Theorem 11.4) shows the initial occurrence satisfies C_0 . $\square$

14 Completeness

Lemma 14.1. *If C_0 is satisfiable then there is an accepted, possibly irregular, witness-generated weak split presentation over the finite profile/context/mosaic alphabet determined by C_0 .*

Proof. Let $\mathcal{I}, a_0 \models C_0$. By Lemma 4.3 restrict to the witness-generated submodel $J = \mathcal{I}|_W$. Give each $a \in W$ its closure type $\text{tp}(a)$. The selected existential witnesses give generated cover edges and containment demands. By Lemma 5.4 these demands have a weak containment split-tree presentation. Record the finitely many profile-level isomorphism classes of local contexts and sibling mosaics that appear, and close this finite set under submosaics, child/context extension, typed-EQ replacement, and triple support. Existential PP/PPI requirements are accepted because selected witnesses were chosen for every true existential closure formula. Hence we obtain an accepted, possibly irregular, presentation over a finite alphabet. $\square$

Lemma 14.2 (Regularization). *If the finite validity rules accept some finitely branching split presentation, then they accept a regular one generated by a finite graph.*

Proof. The validity rules are recognized by the deterministic parity automaton $\mathcal{P}$ constructed above: local network conditions and summary updates are finite transition checks, and transitive existential eventualities are parity conditions. If the automaton accepts some tree, the corresponding finite parity game is winning for the existential player. Finite parity games have memoryless winning strategies; unfolding such a strategy gives a regular accepted tree, hence a finite generator graph. $\square$

Theorem 14.3 (Completeness). *If C_0 is satisfiable then C_0 has a valid finite witness-generated regular split-tree certificate.*

Proof. Lemma 14.1 gives an accepted presentation over a finite alphabet. Regularization yields a regular accepted presentation generated by a finite graph. The finite generator, profiles, contexts, mosaics, initial profile, and parity condition form a valid finite certificate. $\square$

15 Decision procedure

Given C_0 , compute $\text{Cl}(C_0)$ and $\text{Type}(C_0)$. Enumerate finite profile sets, one-hole contexts, atomic sibling mosaics, finite generator graphs, and parity conditions up to computable bounds determined by $\text{Cl}(C_0)$. For each candidate certificate:

1. compute the support-closed mosaic closure (Lemma 10.3);
2. construct the deterministic parity automaton $\mathcal{P}$ for trunks;
3. check parity emptiness on the finite parity game derived from $\mathcal{P}$;
4. verify finite local checks on contexts and mosaics and compute typed congruence.

Effectivity and coarse bounds All certificate components are drawn from finite sets computable from $\text{Cl}(C_0)$. Coarse computable bounds are:

- **Profiles:** at most $2^{|\text{Cl}(C_0)|}$ possible types and hence at most $2^{|\text{Cl}(C_0)|}$ profiles when summaries and monitor subsets are taken into account.
- **Contexts:** bounded by the profile set and interface arity n (in practice $n \leq 3$ suffices); number of contexts is at most $\text{poly}(|\Sigma|, n)$.

- **Atomic mosaics:** bounded by $|\Sigma|^n$ times the finite set of RCC5 labellings on n vertices.
- **Generator graphs:** finite directed graphs over the profile set with size bounded by a function of $|\Sigma|$.

Hence enumeration is effective (computable) though potentially large; each candidate certificate is checked in finite time.

Effectivity and coarse bounds All certificate components are drawn from finite sets computable from $\text{Cl}(C_0)$. Coarse computable bounds are:

- **Profiles:** at most $2^{|\text{Cl}(C_0)|}$ possible types and hence at most $2^{|\text{Cl}(C_0)|}$ profiles when summaries and monitor subsets are taken into account.
- **Contexts:** bounded by the profile set and interface arity n (in practice $n \leq 3$ suffices); number of contexts is at most $\text{poly}(|\Sigma|, n)$.
- **Atomic mosaics:** bounded by $|\Sigma|^n$ times the finite set of RCC5 labellings on n vertices.
- **Generator graphs:** finite directed graphs over the profile set with size bounded by a function of $|\Sigma|$.

Hence enumeration is effective (computable) though potentially large; each candidate certificate is checked in finite time.

Accept iff some valid certificate starts at a profile containing C_0 . Each step is finite and effective; parity emptiness is decidable. Hence satisfiability is decidable.

Effectivity and coarse bounds All certificate components are drawn from finite sets computable from $\text{Cl}(C_0)$. A coarse bound is: number of profiles $\leq 2^{|\text{Cl}(C_0)|}$; number of contexts is bounded by the profile set and interface arity n (typically $n \leq 3$); atomic mosaics are bounded by profiles ^{n} ; generator graphs are finite directed graphs over the profile set. Hence enumeration is effective (though potentially large); each candidate is checked in finite time.

16 Test formulas and worked example

16.1 Chain formula

Consider

$$C_{\uparrow} = \exists \text{PP}. \top \wedge \forall \text{PP}. \exists \text{PP}. \top.$$

A valid certificate representing a satisfying model with no semantic ambient top uses a single profile p with $\exists \text{PP}. \top, \forall \text{PP}. \exists \text{PP}. \top \in \text{tp}(p)$, a one-node generator with a self-loop, trivial mosaics, and a parity condition requiring the existential to be witnessed along the upward trunk. Unfolding the generator yields an infinite upward trunk

$$u_0 \xrightarrow{\text{PP}_{\text{gen}}} u_1 \xrightarrow{\text{PP}_{\text{gen}}} u_2 \xrightarrow{\text{PP}_{\text{gen}}} \dots$$

where each u_i is a fresh occurrence with profile p . The parity acceptance ensures each universal/existential requirement is discharged. Quotienting by typed congruence does not collapse cycle laps; the model satisfies $C_{\uparrow}$.

17 Implementation note

The certificate language above is the formal decision object. An implementation may use a practical checker (CT1–CT5) as an operational approximation; to claim the implementation is a proved decision procedure one must prove CT1–CT5 equivalent to the certificate validity conditions. The formal theorem stands independently of any particular implementation.

18 Concluding remarks

The Option 2 witness-generated regular split-tree architecture repairs the rooted-Hasse assumption and provides a finite certificate language capturing infinite PP/PPI behaviour via ultimately periodic trunks and parity acceptance. The remaining formal components are constructive and effective; the decision procedure is explicit. Complexity may be high in practice, but decidability is established.

Acknowledgements. The document integrates the split-forest ideas with witness-generated repairs and parity encoding to produce a self-contained certificate-based decidability proof.

A RCC5 composition table

Notation. The base relations are EQ, *DR*, *PO*, *PP*, *PPI*. The entry in row R_1 , column R_2 is $\text{comp}(R_1, R_2)$ (the set of possible relations from an element in relation R_1 to a second and R_2 from the second to a third).

Pair (R_1, R_2)	Composition $\text{comp}(R_1, R_2)$
(EQ, EQ)	EQ
(EQ, DR)	DR
(EQ, PO)	PO
(EQ, PP)	PP
(EQ, PPI)	PPI
(DR, EQ)	DR
(DR, DR)	DR
(DR, PO)	DR
(DR, PP)	DR
(DR, PPI)	DR
(PO, EQ)	PO
(PO, DR)	DR
(PO, PO)	PO; DR
(PO, PP)	PO; DR
(PO, PPI)	PO; DR
(PP, EQ)	PP
(PP, DR)	DR
(PP, PO)	PO; DR
(PP, PP)	PP; PO; DR
(PP, PPI)	PO; DR; EQ
(PPI, EQ)	PPI
(PPI, DR)	DR
(PPI, PO)	PO; DR
(PPI, PP)	PO; DR; EQ
(PPI, PPI)	PPI; PO; DR

Remark. This table is the standard RCC5 composition table presented compactly. Cells list all base relations that may result from composition; semicolons separate multiple possibilities. Use this table in the triple-support closure rule: whenever two pairwise mosaics realize R_{12}, R_{13} , the certificate must allow any $R_{23} \in \text{comp}(R_{12}, R_{13})$.

B Worked certificate examples

B.1 Example 1: Upward chain (already sketched)

Concept

$$C_{\uparrow} = \exists \text{PP}. \top \wedge \forall \text{PP}. \exists \text{PP}. \top.$$

Certificate (minimal)

- **Profiles** $\Sigma = \{p\}$ with $\text{tp}(p)$ containing $C_{\uparrow}$, and $\text{Le}(p) = \emptyset$, $\text{Ly}(p) = \text{Cl}(C_0)$ (no upper losses), $\text{M}(p) = \{\exists^+ \text{PP}. \top\}$ (monitor the transitive PP existential), $e(p) = \text{colour}_0$, $s(p) = \text{core}$.
- **Contexts** Ctx_Q : a single one-hole context that allows plugging p under p with vertical label PP (no side branches). The context records $\text{newLower}_C(p, p) = \{\top\}$ and $\text{newUpperSafe}_C(p, p) = \text{Cl}(C_0)$.
- **Mosaics** Mos_Q : trivial one-edge mosaics consistent with PP labelling between occurrences of p .

- **Generator** G_Q : single node labelled p with a self-loop (blocking cycle) that unfolds to fresh occurrences on each traversal.
- **Parity** Π_Q : parity condition requiring that the monitor for $\exists^+ \text{PP}.\top$ is seen infinitely often on the upward trunk; encoded by a simple Büchi set $F_\eta = \{p\}$ and parity indices per Section 7.

Correctness sketch Unfolding the generator yields an infinite upward trunk $u_0 \xrightarrow{\text{PP}_{\text{gen}}} u_1 \xrightarrow{\text{PP}_{\text{gen}}} u_2 \cdots$ with each occurrence typed by p . $\exists \text{PP}.\top$ at each occurrence is witnessed by the child occurrence. The universal $\forall \text{PP}.\exists \text{PP}.\top$ is satisfied because every PP-neighbour (the parent in the upward direction) has a PP-child by the generator loop. The parity condition ensures the transitive eventuality is discharged along the trunk. Quotienting by typed congruence yields a strong RCC5 model satisfying $C_\uparrow$.

B.2 Example 2: Sibling triple requiring triple support

Intended semantic pattern Three occurrences a, b, c arranged so that:

$$a \text{ PP } b, \quad a \text{ PP } c, \quad b ? c$$

and composition requires a compatible relation between b and c (e.g., if both edges are PP then b and c may be PP, PO, DR). This pattern tests the triple-support closure and mosaic construction.

Concepts and profiles Let the closure include a small set of atomic concepts so profiles can distinguish endpoints. Define three profiles:

- p_a with $\text{tp}(p_a)$ containing a concept A and $\text{M}(p_a) = \emptyset$.
- p_b with $\text{tp}(p_b)$ containing B .
- p_c with $\text{tp}(p_c)$ containing C .

Summaries are minimal; equality colours are distinct where needed.

Contexts and atomic mosaics Construct atomic pair mosaics that realize the pairwise relations:

- Mosaic m_{ab} realizing (p_a, p_b) with label PP on the a – b edge.
- Mosaic m_{ac} realizing (p_a, p_c) with label PP on the a – c edge.

Apply the triple-support closure: consult the RCC5 composition table (Appendix A) for $\text{comp}(\text{PP}, \text{PP}) = \{\text{PP}, \text{PO}, \text{DR}\}$. The closure adds three candidate three-endpoint mosaics for (a, b, c) with third edge labels PP, PO, and DR respectively, each checked for relation-safety and typed-EQ consistency.

Generator and acceptance The generator graph G_Q has a root node producing an occurrence of p_a with two outgoing context plugs that create occurrences of p_b and p_c as children (containment occurrences). No parity condition is needed for this finite pattern.

Correctness sketch Because the triple-support closure explicitly added a three-endpoint mosaic for each $R_{bc} \in \{\text{PP}, \text{PO}, \text{DR}\}$, the certificate contains a mosaic that realizes a consistent label for the b – c edge compatible with the two PP edges from a . The typed congruence algorithm then ensures labels are representative-independent; quotienting yields a finite strong RCC5 model with three elements satisfying the intended relations. This certificate therefore witnesses satisfiability of any concept that requires such a local sibling triple (for example, a concept that asserts an A with two distinct PP-children of types B and C).